PRODUCT PROPOSAL · FOR NOTEBOOKLM

# Receipts

## No source, no claim.

Competitive teardowns that show their sources. A complete product proposal — prepared for NotebookLM ingestion.

WORLD PRODUCT DAY 2026 · “EVERYONE SHIPS NOW”
MIND THE PRODUCT × PENDO
github.com/elangovanvp/Receipts
Document version 1.0 · June 2026

# Contents

# 1. Executive Summary

Receipts — competitive teardowns that show their sources. No source, no claim.

### No source, no claim.

That line is the whole product. Receipts is an AI agent that writes competitive teardowns in which every claim links to the exact source page it came from — and that visibly refuses to assert anything it cannot back with a fetched source.

#### The one-paragraph elevator

Paste any product — a competitor, a hot startup, or your own. In under a minute, one AI agent reads its real homepage, pricing page, and changelog, then writes a sharp six-facet teardown — positioning, ICP, pricing, recent moves, strengths, weaknesses — in which every claim links to the exact page it came from. Anything the agent cannot back with a source goes into a visible "Couldn't verify" block, shown openly rather than fabricated.

#### The problem it targets

Product managers have stopped trusting AI for competitive analysis, and they say so plainly. Verbatim pain from r/ProductManagement (2024–2026): "With AI, it's confabulated fairy tales… competitive analysis Kabuki-mime theater certainty we don't really have," and "fed ChatGPT release notes… still got hallucinations and went back to reading them all myself."

Two patterns sit underneath those complaints:

- AI competitive tools confabulate. Confident bullet points, no citations, no dates — no way to separate what is real from what was invented.
- Manual competitive matrices are too slow to survive. They take roughly a month to build, then get "shelved" — stale before they are useful.

Receipts addresses the trust gap directly, trading hallucinated confidence for intel a reader can check.

#### Who it is for

Receipts serves the people the enterprise competitive-intelligence market ignores: solo and early-stage PMs, founders running a pre-build check, and anyone burned by a hallucinated teardown. Incumbents like Crayon, Klue, and Kompyte sell continuous competitor indexing on enterprise contracts (commonly cited around $15k–$30k/year — treat as illustrative) aimed at dedicated CI teams. Receipts does not compete for those teams; it serves the long tail those contracts price out.

#### The core principle: epistemic honesty

One idea holds the product together, and it is a feature rather than a disclaimer: the agent visibly refuses to assert what it cannot source. A hard rule lives in the system prompt — no source  not asserted. Unsourced findings are not quietly dropped; they are routed to a separate, visible "Couldn't verify" block that lists what the agent looked for but could not back. That block is also the human-in-the-loop gesture — instead of auto-asserting, the agent defers judgment to the reader. No incumbent owns this position.

#### The free stack

Receipts runs 100% free, no credit card. Both models run on Groq's free tier — Llama 3.3 70B for the research phase, GPT-OSS 120B for synthesis. Both tools — web search and page fetch — run through Jina, free and keyless. It can optionally swap to Google's Gemini free tier via three environment variables. With no key at all, the app still serves a bundled real Linear sample, so the full experience works. The constraint is the point: accessibility for the audience the paid market abandoned.

### Why this is a shipped product, not a demo

Receipts is live at a public URL a stranger can use right now, with no login. A single Next.js API route streams the agent's real tool calls over Server-Sent Events; share permalinks (`/t/<target>`) auto-run a teardown for cold visitors; Markdown export carries every source link. Novus/Pendo agent analytics are wired in. It deliberately ships one coherent thing — no accounts, no saved workspaces, no payments.

### What makes it distinctive

- Citations are mandatory, not decorative. Every claim carries an "Exhibit" source chip; nothing ships unsourced.
- It admits what it doesn't know. The visible "Couldn't verify" block makes honesty a feature.
- The work is visible as it happens. A cinematic loading state shows the agent's real searches and fetches live — proof it is reading actual pages.
- It is genuinely free and instant. A full sourced teardown in under a minute, at zero cost and no signup.

## 2. The Problem: Why PMs Stopped Trusting AI for Competitive Intelligence

Confident, uncited, and unverifiable — how AI broke the one task PMs hoped it would fix

A product manager on r/ProductManagement put it more sharply than any analyst could. Asked how AI was working for competitive analysis, they said that "with AI, it's confabulated fairy tales" — a "competitive analysis Kabuki-mime theater certainty we don't really have." Another, who tried to do it responsibly, fed ChatGPT the actual release notes and still got hallucinations, until they gave up and "went back to reading them all myself."

That last sentence is the whole problem in miniature. The supposed time-saver sent a careful person back to the manual work they were trying to escape — only slower, because they first had to discover the AI was wrong.

### Three failure modes

1. Confabulation that looks like rigor. Today's AI competitive tools produce output that reads authoritative: tight bullet points, decisive claims, a confident tone. What they almost never produce is a way to check the work. No citation to the page a claim came from. No date telling you whether "they just launched X" happened last week or two years ago. No signal separating what the model read from what it pattern-matched into existence. The format mimics rigor while quietly omitting the one thing rigor requires — a verifiable source. In competitive intelligence, where a single wrong fact about a rival's pricing or roadmap can misdirect a real decision, plausible-but-unsourced is worse than nothing: it spends trust it hasn't earned.

2. Manual matrices that get shelved. The traditional alternative is to do it by hand — open every competitor's site, pricing page, and changelog, and assemble a comparison matrix. Done properly, this can take the better part of a month. By the time it's finished, parts are already stale; a competitor reprices, ships a feature, or repositions, and the document quietly gets shelved. The effort is real, the shelf life is short, and most teams can't justify rebuilding it often enough to keep it true.

3. The collapse of trust. These two failures compound into the most expensive one. Burned by hallucinated teardowns and unwilling to maintain a matrix that decays, PMs default back to the most reliable tool they have: reading every page themselves, by hand. AI was supposed to remove that labor. Instead it added a verification tax on top of it. The people who most need fast, current competitive intel — and who were the natural early adopters of AI for it — have learned not to trust the category at all.

### The painful moment

Picture it concretely. It's the morning of a positioning review, a board check-in, or a sales call where a prospect is weighing you against a named rival. You have fifteen minutes. You ask an AI to brief you on that competitor. It hands back a crisp, confident summary — and you have no way to tell which lines are real. So you do the only safe thing: you open the competitor's site in another tab and start reading, line by line, racing the clock. The tool that promised to save you the work has made you its fact-checker, at exactly the moment you can least afford it.

That is the gap Receipts is built for. The problem was never that AI can't read a competitor's pages — it's that nothing makes the AI prove it did, and refuse to assert what it couldn't source. Competitive intelligence didn't break because the analysis is hard. It broke because the confidence stopped being trustworthy. The fix isn't a smarter model; it's a stricter rule — no source, no claim — and a product built to honor it in the open.

## 3. The Insight: Epistemic Honesty as a Product

The non-obvious bet — don't make the AI sound more confident, make it provably honest.

Most AI tools compete on a single axis: sounding more authoritative. They smooth over uncertainty, round up to confident bullet points, and use the same even tone whether a line was read off a real pricing page or invented whole. Receipts makes the opposite bet. It does not try to sound more confident. It tries to be provably honest — citing every claim it makes, and openly showing what it could not verify.

That inversion is the whole product.

### Why showing the gap beats filling it

A competitive teardown is only useful if the reader can act on it. The reason product managers stopped trusting AI for this job is not that AI gets everything wrong — it is that they can no longer tell which parts are wrong. As one practitioner put it on r/ProductManagement, the output reads like "competitive analysis Kabuki — mime theater certainty we don't really have." Confidence without provenance is worse than useless, because it costs the reader the exact effort the tool promised to save: re-verifying everything by hand.

Receipts treats the unknown as information, not failure:

- A filled gap hides risk. When an agent fabricates a plausible line to complete the picture, the reader inherits a landmine they cannot see.
- A shown gap transfers control. The visible "Couldn't verify" block tells the reader exactly where to look next. It is a research to-do list, not an excuse.
- An honest gap is checkable. Because every asserted claim links to the source page it came from, a skeptical reader can spot-check in seconds rather than redo the work.

This is also the human-in-the-loop gesture. The agent does not assert past the edge of its evidence; it defers judgment to the user. That deference is what earns the trust back.

### "No source, no claim" as constraint and brand

The doctrine is enforced, not aspirational. It lives as a hard rule in the synthesis system prompt: no source, not asserted. During writing, any item the agent looked for but could not back with a fetched page is routed out of the six facets and into the visible unverified block. The same rule shapes the data model — every `Claim` carries a `source` with a title, URL, and optional quote, and confidence is graded by corroboration: "high" means two or more sources agree, "medium" means a single source.

Because the rule is structural, it doubles as the brand. The rallying line — "No source, no claim" — is not marketing bolted on afterward; it is a literal description of how the system behaves. The design system reinforces it: receipt-style "Exhibit" chips stamp each claim with its source, an amber highlighter lands only on corroborated claims, and redaction bars mark the things the agent honestly could not confirm. The promise and the product are the same object.

### Why no incumbent owns this position

The established competitive-intelligence tools — Crayon, Klue, Kompyte (typically enterprise contracts, often cited in the roughly $15k–$30k/year range; treat as illustrative) — compete on coverage and continuous indexing for enterprise CI teams. The consumer-grade AI tools compete on fluency. Neither competes on refusing to assert what it cannot source, because for both, visible uncertainty looks like a weakness to hide.

That leaves the honesty position genuinely open. It is hard to occupy precisely because it runs against the grain of what makes AI demos impressive — you have to be willing to show less, on purpose. Receipts is built for the people that market ignores: solo and early-stage PMs, founders running a pre-build check, and anyone burned once by a hallucinated teardown. For them, "shows its sources, admits what it doesn't know" is not a constraint to apologize for. It is the only version worth using.

# 4. Market Landscape and Willingness to Pay

The paid competitive-intelligence market serves enterprise teams. Receipts serves the people that market was never priced for.

A paid market for competitive intelligence already exists. So the question Receipts answers is not "will anyone pay for this?" — clearly some buyers already do — but "who is locked out of it, and what would they actually reach for instead?"

### The incumbent market: continuous indexing for enterprise CI teams

A category of well-funded tools already tracks competitors continuously. The names most often cited are Crayon, Klue, and Kompyte, and their model is roughly the same:

- They continuously index competitor websites, pricing changes, ad spend, reviews, and news, building an always-on intelligence feed.
- They are sold to dedicated competitive-intelligence and product-marketing teams inside larger organizations.
- They are priced as annual enterprise contracts, with figures commonly cited in the ~$15,000–$30,000 per year range. (Treat these numbers as illustrative and approximate. They vary by seat count, modules, and negotiation, and are used here only to convey order of magnitude — not as verified quotes.)

This is a legitimate, valuable category for the buyer it was built for: a company with a CI function, a budget line, and an ongoing need to monitor a fixed set of named rivals over quarters and years. Receipts does not compete for that buyer, and it should not pretend to.

### The gap they leave

The enterprise model has two structural consequences, and together they create an underserved population:

- The price floor excludes individuals. A solo product manager, an indie founder, or an early-stage team will not approve a five-figure annual seat to answer one question this week. The minimum viable transaction in that market is far larger than the need.
- Continuous indexing is the wrong shape for a one-off question. Someone running a pre-build sanity check — "is this space already crowded, and how is the leader actually positioned?" — does not want a subscription that watches competitors forever. They want one honest, current read, right now, and then to move on.

So the people left out are not a niche. They are solo and early-stage PMs, founders doing a pre-build check, and anyone who has been burned by a hallucinated AI teardown and now distrusts the fast option. The paid market skips them because they are individually unprofitable to serve under an enterprise sales motion. The free AI tools "serve" them — but with the confident, uncited, undated output this exact audience has already learned not to trust (see the Problem section). That is the open lane.

### Willingness to pay, framed honestly for a hackathon build

Receipts is a hackathon project, and this proposal will not invent demand or revenue. What can be stated soberly:

- The wedge is accessibility, not a price tag. The entire stack is deliberately free and keyless (Groq, Jina), so the product can be handed to a stranger with no login and no card. That is a product and World Product Day eligibility decision, not a monetization claim.
- Realistic future willingness-to-pay is small and individual, not enterprise. To a solo PM, a single trustworthy, sourced teardown is worth closer to the price of a good report than a CI platform seat. The honest near-term "currency" is usage and sharing, captured by the share/export goal event — not dollars.
- It targets a problem people already pay to solve. Judges weighing willingness-to-pay (for example, Amit Godbole) can note that demand is proven by Crayon, Klue, and Kompyte's existence; Receipts simply addresses the slice their pricing structurally excludes.

In short: the spend is real, the gap is real, and Receipts' claim is modest and specific — be the instant, honest, one-off read for the people the enterprise CI market was never priced to serve.

# 5. Target Users & Personas

The people the enterprise competitive-intelligence market was never priced for

Receipts is built for the people Crayon, Klue, and Kompyte don't sell to. Those tools continuously index competitors and close annual enterprise contracts (commonly cited in the ~$15k–$30k/year range; treat as illustrative), aimed at dedicated competitive-intelligence teams. The personas below are the wedge into the audience that market overlooks: solo and early-stage PMs, founders running a pre-build check, and anyone who has been burned by a hallucinated teardown. Each has a sharp moment of need, a frustrating status quo, and one specific thing Receipts changes.

### Persona 1 — Priya, the solo PM prepping for a meeting

Moment of need. It's the night before a roadmap review. Priya is the only PM at a 30-person company, and someone will ask, "How does our pricing compare to Competitor X?" She needs an answer she can defend in the room, not a vibe.

What she does today. She opens ChatGPT, asks for a competitive breakdown, and gets confident bullet points with no citations and no dates. She can't tell which lines are real, so she opens fifteen browser tabs and rebuilds it by hand — an hour she didn't have.

What Receipts changes. She pastes the competitor's name and, in under a minute, gets a six-facet teardown where every claim links to the exact page it came from — the homepage, the pricing page, a changelog entry. In the meeting she doesn't say "I think"; she clicks the source. The "Couldn't verify" block tells her precisely which questions are still open, so she walks in knowing the edges of her own knowledge.

### Persona 2 — Marco, the founder doing a pre-build market check

Moment of need. Marco is about to commit three engineers to a feature. First he wants to know what the obvious incumbent already ships and where it's genuinely weak — today, before the sprint is planned.

What he does today. He skims the competitor's marketing site, half-trusts an AI summary, and guesses at the gaps. A full manual competitive matrix would take a month and then get shelved, so he builds on a hunch and finds out later.

What Receipts changes. Receipts reads the real homepage, pricing, and changelog, plus one credible third-party review or "limitations/complaints" page — the source Exhibit F (Weaknesses) is built from. Marco gets Recent moves (what shipped) and Weaknesses (corroborated complaints), each with an openable source. He can tell a high-confidence claim (corroborated by two or more sources) from a single-source medium one, and decide on evidence. The Markdown export carries every source link straight into his planning doc.

### Persona 3 — Dana, the PM burned by a hallucinated teardown in Slack

Moment of need. Dana once pasted an AI teardown into a Slack channel. A teammate caught that a "feature" the model confidently described didn't exist. The credibility hit stuck, and now she distrusts AI for competitive work entirely — the verbatim Reddit complaint: "confabulated fairy tales… competitive analysis Kabuki-mime theater certainty we don't really have."

What she does today. She reads every release note herself, exactly as one r/ProductManagement commenter described after AI "still got hallucinations." Reliable, but it doesn't scale, and it's the work she most resents.

What Receipts changes. The hard rule — no source, no claim — means nothing reaches her teammates unless a fetched page backs it. Anything the agent can't verify is shown openly in a separate block, never invented. The cinematic loading state surfaces the agent's real searches and fetches as they happen, so she watches it read actual pages. For the first time, Dana can paste a teardown into Slack and stand behind every line — because each one carries a receipt.

These three share one trait: they value trust over volume. That is the willingness-to-pay Receipts targets — the honesty the incumbents don't sell.

# 6. Product Overview: What Receipts Does

One screen, no login, a sourced teardown in under a minute.

Receipts does one thing completely: you paste a product, and an AI agent hands back a competitive teardown where every claim links to the page it came from. No setup, no account, no dashboard to configure. The entire experience lives on a single screen.

### The end-to-end behavior at a glance

The flow is deliberately linear, so a first-time visitor never has to learn an interface:

- Paste a product. Type a competitor, a hot startup, or your own product into one input. That is the entire "configuration" step.
- The agent searches. It runs real web searches (via Jina's keyless search) to find the product's official surfaces.
- The agent fetches real pages. It reads the actual homepage, a pricing page, a changelog or release-notes page, and at least one credible third-party review or "limitations/complaints" page — typically three to five fetches in all. The reader returns clean page text, not a recollection from training data.
- The agent writes a six-facet teardown. Synthesis produces a structured case file organized as Exhibits A–F: Positioning, ICP (who it's for), Pricing, Recent moves, Strengths, and Weaknesses. Each facet is a short list of sharp, one-sentence claims.
- Every claim carries a source chip. No claim appears without a linked source — title, URL, and often a supporting quote. Claims corroborated by two or more sources are marked high-confidence; single-source claims are marked medium. This is the product's core rule, stated plainly: no source, no claim.

- Unsourced items go to "Couldn't verify." Anything the agent looked for but could not back with a fetched page is listed openly in a separate block, with the reason. It is never silently dropped and never fabricated to fill a gap.
- Share or export. When the teardown is done, you can copy a permalink, export to Markdown (every source link travels with it), or share. That ends the flow.

### Why the "Couldn't verify" block matters

Most AI competitive tools assert confidently and cite nothing. Receipts inverts that posture. The "Couldn't verify" block is not an error state — it is the product working as designed. It is the agent visibly deferring to the user instead of auto-asserting, and it is what makes the rest of the report trustworthy: an agent willing to admit what it doesn't know earns more weight for the claims it does make.

### The single screen

Receipts has no navigation, no settings panel, and no workspace. One screen handles every state — idle (the input), running (a live view of the agent's real tool calls as they happen), done (the finished case file), and error. While the agent works, it streams its actual searches and fetches in row by row, so the claim "I am reading real pages right now" is shown rather than asserted.

### The no-login promise

A stranger can use Receipts immediately. There are no accounts, no email gates, no saved workspaces. Share permalinks at `/t/<target>` auto-run the teardown for a cold visitor, so a shared link works on the first click. Permalinks are the only persistence the product keeps — a deliberate scope choice, not an oversight.

### The sub-minute target

The whole loop — paste, search, fetch, synthesize, render — is designed to finish well inside the API route's 60-second budget, with roughly 45 seconds as the working target. Results stream in word by word as they are written, so the page fills with sourced claims while the agent is still working, instead of holding the reader on a spinner for one final payload.

## 7. The User Journey: Land to Value to Share

One target in, a sourced teardown out — in under a minute, with nothing hidden.

Receipts runs on a single linear path. There is no onboarding, no account, no workspace to configure. A first-time visitor and a returning one travel the same short road: land, submit a target, watch the agent work, read the teardown, share it. Each step is built so the product's core promise — no source, no claim — is visible rather than asserted.

### 1. Land on a single intake

The landing page is the product. The hero presents the 3D tilting dossier card — folder tab, cursor-reactive wax seal — and beneath it one input: paste any product. A competitor, a hot startup, your own. There is no menu of options and no settings to weigh; the only decision is what to tear down. This is where the funnel opens: the `landed` step is recorded here.

### 2. Submit a target

The user types or pastes a target and submits. This fires `trackAgent("prompt", { target, surface })` and marks `submitted_target` in the funnel. A single API route, `POST /api/teardown`, opens a Server-Sent-Events stream, and the view transitions (via AnimatePresence) from idle to running. No page reload, no spinner-then-blank wait.

### 3. Watch the agent gather evidence — live

Instead of a generic loading state, the CaseFileGathering view shows the agent's real work as it happens. This is the most important moment in the journey, because it is where the central claim — "I am reading real pages right now" — becomes literally observable:

- A pulsing clay dot signals active work.
- A WEB_SEARCH / FETCH activity log slides in row by row, naming the actual queries run and URLs fetched.
- A phase indicator moves SEARCHING → FETCHING → SYNTHESIZING along animated connector lines.

Under the hood the agent emits one line at a time — SEARCH: <query> or FETCH: <url> — and the server runs each tool (Jina search and reader) and returns results. It aims for three to five fetches: the official site, a pricing page, a changelog or release-notes page, and at least one third-party review or "limitations/complaints" page. The user watches that sequence unfold rather than a progress bar pretending to be busy.

### 4. The teardown renders, facet by facet

As synthesis begins, claims stream in. Each claim reveals word by word and lands inside its facet — the six case-file Exhibits A–F: Positioning, ICP, Pricing, Recent moves, Strengths, Weaknesses. Every claim carries a receipt/stamp "Exhibit" chip linking to the exact source page it came from. High-confidence claims, corroborated by two or more sources, get an amber highlighter swipe; single-source, medium-confidence claims appear plainly. Claims arrive incrementally over SSE (`claim` events), so the dossier fills in front of the reader. This is `teardown_rendered`.

### 5. The "Couldn't verify" block

Below the facets sits a separate, visibly distinct block rendered with redaction bars. It lists what the agent looked for but could not back with a fetched source. This is the human-in-the-loop gesture: the agent defers judgment to the reader instead of inventing a confident answer. Showing the gaps openly is what makes the rest of the dossier trustworthy.

### 6. Copy, export, share — goal completion

The journey's goal is not reading; it is sharing what you can stand behind. The ShareBar offers copy-link, export-to-Markdown, and share. The Markdown export carries every source link, so the teardown survives outside the app. This fires `track("teardown_shared_or_exported", { action, target })` — the final funnel step.

#### The permalink that closes the loop

Every teardown lives at a `/t/<target>` permalink. A cold visitor opening that link — no login — triggers the teardown to auto-run for them. Permalinks are the product's only persistence, and its primary path to a new `landed` event.

# 8. The Six-Facet Teardown Framework (Exhibits A-F)

One fixed case file, six exhibits, every entry sourced

A teardown is only useful if it is built the same way every time. Receipts standardizes every report into six facets, labelled Exhibits A through F. The fixed shape lets two teardowns be read side by side, and it forces the agent to hunt for the same kinds of evidence on every run rather than free-associating around whatever the homepage happens to emphasize.

### The "Exhibit" case-file framing

The framing borrows from how a case file is assembled: a claim is not an opinion, it is evidence, and evidence is tagged and admitted under an exhibit number. Each facet carries a letter (A-F), and every claim inside it streams in wearing a receipt-style "Exhibit" chip that links back to the exact page it came from. This is not decoration. It is the product's central rule made visible in the interface — the same discipline the system prompt enforces internally ("no source, no claim"), expressed as a stamped, traceable record. The reader can audit any line by following its exhibit back to the fetched source.

### The six facets and what the agent looks for

Exhibit A — Positioning. How the product frames itself: the tagline, the category it claims, the primary promise. Sourced from the official homepage. It comes first because positioning is the vendor's own self-description, and reading it up front lets every later facet be checked against the company's stated intent.

Exhibit B — ICP (who it's for). The ideal customer or target user — named segments, roles, company sizes, or use cases the product addresses. Drawn from the homepage and any "use cases," "customers," or "who it's for" pages. It matters because a competitor's strength against one audience can be irrelevant to another; knowing the ICP keeps the comparison fair.

Exhibit C — Pricing. Plans, tiers, and what is free versus paid. Sourced from the pricing page, one of the agent's targeted fetches. Pricing is the fastest signal of who a product is really for and where it sits in the market, and the vendor publishes it plainly — so it should be sourced, not guessed.

Exhibit D — Recent moves. What shipped recently, taken from a changelog or release-notes page. This facet captures momentum and direction — exactly the area AI tools most often get wrong by inventing plausible-sounding features. Anchoring it to a dated changelog keeps the intel current and verifiable.

Exhibit E — Strengths. Where the product is genuinely strong, stated as claims the agent can corroborate. Strengths are held to a higher bar: a claim reaches "high" confidence only when two or more sources agree, which is why the amber highlighter lands on corroborated lines. Single-source strengths are kept but marked "medium."

Exhibit F — Weaknesses. Real limitations — deliberately not sourced from the vendor's own site, because no homepage advertises its flaws. The agent reserves one of its fetches for a credible third-party review or a "<product> limitations / complaints" page. This is the facet that most separates Receipts from tools that paraphrase marketing copy: weaknesses come from people outside the company.

### Why six, and why this order

The facets move from how a product presents itself (A, B), to what it costs and what it has done (C, D), to an evaluated verdict (E, F). Anything the agent set out to fill but could not back with a fetched source is neither quietly omitted nor invented — it is routed to the visible "Couldn't verify" block. That routing is what makes the six exhibits trustworthy: the report shows its gaps as plainly as its findings.

## 9. The "Couldn't Verify" Doctrine

No source, no claim — why a visible block of admitted gaps is what makes the rest of the report trustworthy

Every AI competitive tool hits the same fork when it runs out of evidence: invent a plausible-sounding claim, or quietly drop the question. Receipts takes neither path. When the agent looks for something and cannot back it with a fetched source, the finding is neither fabricated nor silently discarded — it is routed to a visible "Couldn't verify" block and shown to the reader. This is the rule the whole product is built around, and the reason the rallying line is "No source, no claim."

### The hard rule

The constraint lives in the synthesis system prompt as an absolute, not a preference: no source, no claim. During the writing phase, the GPT-OSS 120B model emits a strict structured object, and the enforcement is mechanical rather than stylistic. A sentence can only become a `Claim` if it carries a `source` with a real fetched `title` and `url`. Anything the model "knows" but cannot attach to a page it actually read is not softened into a hedged claim — it is moved out of the assertion path entirely, into the `unverified` array as an `Unverified` item, optionally carrying a `reason`.

This matters because the failure mode it prevents is specific and documented. In the brief, product managers describe AI competitive analysis as "confabulated fairy tales" and "Kabuki-mime theater certainty we don't really have." One reports feeding ChatGPT release notes, still getting hallucinations, and going back to reading them all by hand. The doctrine is the structural answer: the model is never given a slot to launder an unsourced guess into a confident bullet.

### Routing gaps instead of hiding them

The six facets (Exhibits A–F: Positioning, ICP, Pricing, Recent moves, Strengths, Weaknesses) hold only what the agent could substantiate. The gaps go somewhere the reader can see them:

- Dropped findings are not lost. If the agent searched for pricing but no pricing page was fetchable, that absence is recorded — "Pricing tiers not stated on a fetched page" — rather than omitted.
- Invention is structurally impossible. Because a `Claim` cannot exist without a `source`, there is no field for an unsourced assertion to be written into.
- The block reflects scope. It shows what the agent looked for, so the reader sees the shape of the investigation, not only its hits.

In the Dossier design language, this block is rendered with redaction bars — a deliberate signal that something is withheld pending evidence, not erased.

### The human-in-the-loop gesture

The "Couldn't verify" block is where the agent defers judgment to the reader instead of auto-asserting. When evidence runs out, a typical tool resolves the ambiguity itself, usually by guessing confidently. Receipts hands the open question back. The reader, who has context the agent lacks, decides whether an unverified item is worth chasing. That deference is the human-in-the-loop posture expressed in the artifact, not buried in a settings panel.

### Mapping to confidence

For the claims that do survive, the same discipline produces two confidence levels:

- High — corroborated by two or more sources. In the UI these earn the amber highlighter swipe.
- Medium — supported by a single source. Real and cited, but standing on one leg.

So a claim's standing is always legible: corroborated, single-sourced, or unverified. There is no fourth bucket for "asserted on vibes."

### Why honesty is the feature

This is the wedge. The product trades hallucinated confidence for verifiable, current intel, and the visible refusal to assert is the proof that the rest of the report can be trusted. A teardown that admits its gaps is more useful than one that paints over them, because the reader can act on the admission. In a category where incumbents confabulate, epistemic honesty is not a shortfall to apologize for — it is the position no one else has taken.

# 10. System Architecture

One streaming route, one server-side agent, two free tools — everything in service of "No source, no claim."

Receipts is deliberately small. There is no microservice fleet, no job queue, no database of tracked competitors. The entire product is a single Next.js 16 application (App Router, TypeScript) deployed on Vercel, with one streaming API route and one server-side agent loop. That narrowness is the design, not a shortcut: it keeps the system auditable, cheap enough to run on free tiers, and fast enough to return a full teardown inside a 60-second request.

## The single route: POST /api/teardown

All work happens behind one endpoint.

- Runtime and limits: `runtime = "nodejs"`, `maxDuration = 60`. Node (not Edge) is required because the agent makes outbound HTTP calls to Jina and Groq during the request.
- Transport: Server-Sent Events (SSE). The route opens a stream and emits typed `StreamEvent` messages as the agent works, so the browser renders claims as they land rather than waiting for one final blob.
- Input: a single `target` string (a product name or URL), plus a `surface` field used only for analytics.

## The agent loop

One server-side agent drives the entire run. To stay model-agnostic, it uses a plain-text protocol instead of native tool-calling, because open models (Llama on Groq) frequently mis-format native tool-call JSON. The model must reply with exactly one line at a time:

- `SEARCH: <query>` — the server runs a web search and returns results.
- `FETCH: <absolute url>` — the server fetches clean page text and returns it.

The server executes the tool, appends the result to the conversation, and asks the model for its next line. The loop targets three to five fetches: the official site, a pricing page, a changelog or release-notes page, and at least one credible third-party review or "<product> limitations/complaints" page — that last fetch is where genuine weaknesses (Exhibit F) come from. When the model judges it has enough evidence, it stops.

### Two models, two phases

- Research and gathering: Llama 3.3 70B (`llama-3.3-70b-versatile`).
- Synthesis and writing: GPT-OSS 120B (`openai/gpt-oss-120b`).

Both run free on Groq. Synthesis emits a strict structured object, and the "no source, no claim" rule is enforced there: any item the agent could not back with a fetched source is routed to the Couldn't verify block rather than asserted.

### Two tools, both Jina

The agent has exactly two tools, both free and keyless: `web_search` via `s.jina.ai`, and `fetch_url` via the `r.jina.ai` reader, which returns clean page text. No scraping infrastructure, no API keys.

## The StreamEvent contract

The SSE stream carries six event types, consumed in order by the browser:

- `status` — phase signal (searching, fetching, synthesizing) that drives the live loading state.
- `meta` — target metadata (canonical URL, tagline) once resolved.
- `claim` — one sourced claim, with its facet and source link, revealed word-by-word in the UI.

- `unverified` — an item the agent looked for but could not source, with an optional reason.
- `done` — terminal success; the teardown is complete.
- `error` — terminal failure with a message.

### End-to-end data flow

1. The browser POSTs `{ target }` and opens the SSE stream.
2. The route starts the agent; `status` events report each real tool call live.
3. As Llama gathers evidence, the server emits `meta`; then GPT-OSS synthesis streams `claim` and `unverified` events.
4. The route closes with `done` (or `error`).

### Persistence and the client

The only persistence is an optional, tiny store for `/t/<target>` share permalinks — no accounts, no saved workspaces. The browser UI carries the Pendo (Novus) SDK, whose calls are safe no-ops until `window.pendo` exists. With no API key configured, the app serves a bundled real Linear sample so the full UI still works.

## 11. The Agent Loop in Detail

A model-agnostic text protocol, a disciplined fetch budget, and a two-model split that separates reading from writing.

---

The agent is the part of Receipts that earns the rallying line. Everything visible in the UI — the Exhibit chips, the highlighter swipe, the redaction bars — is downstream of one server-side loop that reads real pages and refuses to assert what it cannot source. This section explains how that loop works and why it is built the way it is.

### Why a text protocol instead of native tool-calling

Most agent frameworks lean on native function-calling: the model returns a structured JSON tool call, the runtime parses it, runs the tool, and feeds the result back. That works well on frontier closed models. It is unreliable on the open models Receipts runs on for free.

The research phase uses Llama 3.3 70B on Groq, and open models on Groq frequently mis-format native tool calls — malformed JSON, hallucinated argument names, calls wrapped in prose that the parser chokes on. Every malformed call is a wasted turn inside a 60-second budget.

So Receipts sidesteps native tool-calling and uses a plain text protocol instead. The contract is intentionally narrow: the model emits exactly one line per turn, and that line is either

- `SEARCH: <query>` — run a web search, or
- `FETCH: <absolute url>` — fetch and read one page.

The server parses that single line, runs the matching tool, and returns the result as the next message; the model then emits its next line. There is no JSON to malform, no schema to violate — two verbs and a string. A one-line grammar is something even a smaller open model formats correctly turn after turn, which is the entire point: reliability bought by lowering what the model has to produce.

### The fetch loop and its budget

The loop aims for 3–5 FETCHes, and the targets are chosen to cover the six facets:

- the official site (positioning, ICP),
- a pricing page (plans, what is free versus paid),
- a changelog or release-notes page (recent moves), and
- at least one credible third-party review or a `<product> limitations/complaints` page.

That last fetch is load-bearing. Strengths can be corroborated from a vendor's own site, but honest weaknesses rarely live there — they come from outside reviews and complaint threads. Without that fetch, Exhibit F is either empty or, worse, invented. The loop stops when it has enough to fill the facets; it is not padded to hit a number.

Both tools run through Jina — `s.jina.ai` for search and the `r.jina.ai` reader for clean page text — which is free and keyless. The model never sees raw HTML; it reads readable text, which keeps it on task and inside its context budget.

### The two-model split

Receipts uses two models for two different jobs, both free on Groq:

- Research / gathering — Llama 3.3 70B (`llama-3.3-70b-versatile`). This phase decides what to search and fetch next and tolerates noisy page text. It rewards a fast, capable model that follows the one-line protocol.
- Synthesis / writing — GPT-OSS 120B (`openai/gpt-oss-120b`). Once the pages are gathered, a separate pass writes the structured teardown.

The split exists because reading and writing fail in different ways. Letting one model both browse and compose tends to blur sourcing — claims drift away from the pages that justify them. Separating the phases lets the synthesis model work only from gathered text and enforce "no source, no claim" cleanly: every `Claim` carries its `source`, anything that cannot be backed is routed to the visible "Couldn't verify" block, and confidence is marked `high` only when two or more sources corroborate it. Both model IDs are reported to Novus via `agentModelsUsed`, so the Models-used chart renders the split honestly.

## 12. The Free AI Stack

Why running on free, no-card infrastructure is a product decision, not a budget constraint

Receipts runs entirely on free infrastructure, with no credit card required to operate it. That is not a corner cut to save money during a hackathon. It is the same kind of decision as the "Couldn't verify" block: both are about who the product is for.

### Accessibility is the point

The paid competitive-intelligence market — Crayon, Klue, Kompyte — sells into enterprise CI teams on annual contracts (commonly cited in the ~$15k–$30k/year range; treat as illustrative). Those tools continuously index competitors, which is real work that real teams pay for. But the gate is steep, and it prices out exactly the people who feel the pain most acutely: solo and early-stage PMs, founders running a pre-build check, and anyone who got burned by a hallucinated teardown and gave up on AI for the task.

A free, no-login, no-card tool meets those users where they are. Here the cost structure is the access strategy. If a stranger has to reach for a corporate card before they can paste a competitor's URL, the wedge is gone.

### What's in the stack

Receipts uses two language models and two tools, all on free tiers.

- Two models, both free on Groq. The research and gathering phase runs on Llama 3.3 70B (`llama-3.3-70b-versatile`) — fast and good enough to decide what to search and fetch next. The synthesis and writing phase runs on GPT-OSS 120B (`openai/gpt-oss-120b`), a larger model for the harder job of turning fetched pages into sharp, sourced claims. The split shows up in the Novus "models-used" chart, and it reflects a real engineering choice: spend the bigger model only where writing quality decides the outcome.
- Two tools, free and keyless via Jina. Web search goes through `s.jina.ai`; page reading goes through the `r.jina.ai` reader, which returns clean page text instead of raw HTML. Neither needs an API key, which removes both a setup step and a failure mode.
- Optional Gemini swap. A reader who prefers Google's free tier can switch the model layer by setting three environment variables — `LLM_BASE_URL`, `LLM_MODEL`, and `LLM_API_KEY`. No code changes.
- A bundled real sample. With no key configured at all, the app serves a real Linear teardown, so the full UI — streaming, exhibits, highlighter, redaction bars — works on first load. A cold visitor sees the actual product, not an error.

### The engineering trade-offs

Building on free infrastructure is an honest constraint, and it shapes the design rather than hiding behind it.

- Open models mis-format native tool calls. This is the core reason the agent uses a deliberately model-agnostic text protocol: the model replies one line at a time, `SEARCH: <query>` or `FETCH: <url>`, and the server runs the tool. A paid frontier model with reliable native tool-calling might not need this. Designing for Llama on Groq forced a more robust, more portable loop.
- Rate limits and latency are real. Free tiers throttle. The agent counters this by aiming for a tight 3–5 fetches per run and stopping once it has enough, which keeps a teardown inside the 60-second `maxDuration` budget instead of fanning out indefinitely.
- Availability is not guaranteed. A free dependency can change terms or go down. The three-variable model swap and the bundled sample are the hedges: the product degrades to something usable rather than breaking.

The result is a tool that is free because its users should be — and one whose architecture is more portable for having been built under that rule.

# 13. Design System: "The Dossier"

A detective case-file aesthetic for a product whose entire promise is evidence.

The visual language of Receipts is called "The Dossier." It borrows the look of a detective's case file because the product does a detective's work: it gathers material, marks each piece as an exhibit, and refuses to assert anything it cannot back. The aesthetic is not decoration layered onto a generic AI tool — it is part of the argument. A case file implies provenance. Every claim sits next to where it came from, and anything unproven is set aside in plain view rather than smuggled into the narrative. The form mirrors the doctrine: "No source, no claim."

### Why a case file fits a product about evidence

Most AI competitive tools present as clean dashboards that imply authority they have not earned — confident bullets with no citations, no dates, no chain of custody. The Dossier deliberately inverts that. It looks handled, annotated, and accountable. The reader is invited to inspect the file, not just trust the summary. That posture is the wedge: the interface should feel like it is showing its work, because the product is.

### Palette and type

The palette is warm and analog, evoking paper, ink, and physical marks:

- Clay #C25A38 and clay-deep #A8472A — the primary accent and active states.
- Warm paper #FFFDF8 — the card surface, like a folder page.
- Hairline #E6DDCC — fine dividers that read as ruled lines.
- Ink and ink-secondary #6E675B — primary and muted body text.
- Wax #B23A2C — the seal and stamp red.
- Amber — the highlighter that lands only on corroborated claims.

Type pairs three faces for three jobs: Fraunces (serif display) for headlines and the editorial voice; Inter (body) for readable prose; Geist Mono (labels and code) for exhibit tags, URLs, and the agent's tool log. The serif-plus-mono contrast reinforces the product's central tension: a human-readable narrative laid over a machine-verified record.

### Signature elements

- The 3D tilting dossier card. The hero is a folder-tab card that tilts subtly toward the cursor, with a cursor-reactive wax seal that catches the light as the pointer moves. It establishes the metaphor before a word is read.
- Exhibit chips. Every claim carries a stamped chip naming its facet — Exhibit A through F — and linking to its source, so provenance is attached to the claim rather than buried in a footnote.
- The amber highlighter. A swipe of amber animates across corroborated, high-confidence claims — those backed by two or more sources. The mark is earned, never default; it visually separates strong intel from single-source notes.
- Redaction bars. The "Couldn't verify" block renders as redaction bars: the agent admits, in plain sight, what it looked for and could not source. Honesty is made literal.
- Word-by-word reveal. Each claim surfaces word by word as it streams in, so the reader watches the file being written.
- The "gathering evidence" loading state ("CaseFileGathering"). Instead of a generic spinner, it surfaces the agent's real tool calls live — a pulsing clay dot, a WEB_SEARCH / FETCH activity log that slides in row by row, and a SEARCHING → FETCHING → SYNTHESIZING phase indicator with animated connectors. It makes the core claim — "I am reading real pages right now" — visible.

### Motion and reduced-motion safety

Motion uses Lenis smooth scroll, scroll-driven parallax on the hero glow, and AnimatePresence transitions between the idle, running, done, and error views. Every effect is reduced-motion safe: under `prefers-reduced-motion`, the experience falls back to opacity-only changes, and the tilt, parallax, and magnetic-seal effects are fully disabled. The atmosphere is a layer, never a barrier to reading the evidence.

# 14. Measurement and Novus Instrumentation

What Receipts tracks, why it maps to honest product signals, and why a tool-heavy agent reads as genuinely agentic in Novus

Novus, Pendo's agent-analytics layer, is mandatory for the World Product Day 2026 entry — projects without it are ineligible. But Receipts does not treat it as a compliance checkbox. It is the layer that lets the product prove, in data, the claim it makes in prose: that a real agent reads real pages and refuses to assert what it cannot source. The instrumentation is designed to tell that story accurately, not to inflate it.

### The three events

Receipts emits exactly three tracked events, each mapped to a moment that matters.

- `trackAgent("prompt", { target, surface })` fires the instant a teardown starts — the moment the user submits a product. `target` is the product name or URL; `surface` distinguishes the main app from a `/t/<target>` permalink. This is the agent's first turn.
- `trackAgent("agent_response", { target, status, toolsUsed, toolCalls, claims, unverified, agentModelsUsed })` fires on completion, for both `done` and `error`. It carries the run's full shape: `status`; `toolsUsed`, a breakdown of `web_search` and `fetch_url`; `toolCalls`, the raw count of tool invocations; `claims`, the number of sourced claims synthesized; `unverified`, the number of items routed to the "Couldn't verify" block; and `agentModelsUsed: ["llama-3.3-70b-versatile", "openai/gpt-oss-120b"]`.
- `track("teardown_shared_or_exported", { action, target })` is the goal event, firing on copy-link, export-to-Markdown, or share. `action` records which.

Every call is a safe no-op until `window.pendo` exists, so the app runs identically with or without the SDK. The Novus install PR adds it.

### The funnel

The four-stage funnel is deliberately legible:

`landed` → `submitted_target` → `teardown_rendered` → `teardown_shared_or_exported`

Each stage maps to an observable user intent: arrival, committing a target, receiving a finished case file, and finding it useful enough to keep or pass on. Because share permalinks auto-run for cold visitors, a shared link re-enters the funnel at the top — sharing is both a goal event and an acquisition path.

### Why a tool-heavy agent reads as genuinely agentic

The agent aims for three to five FETCHes plus searches per run: the official site, a pricing page, a changelog, and at least one third-party review or limitations page. Each is a discrete, logged tool call, surfaced both to the user — in the live `CaseFileGathering` log — and to Novus, via `toolCalls` and `toolsUsed`. So the Conversations, Tools-used, and Models-used dashboards fill with real activity rather than a single opaque LLM call.

The two-model split makes this honest rather than decorative. Research runs Llama 3.3 70B; synthesis runs GPT-OSS 120B. The Models-used chart renders two distinct models doing two distinct jobs — gathering and writing — which is an accurate picture of the architecture, not a staged one. A reviewer reading the dashboard sees a multi-step, multi-tool, multi-model loop because that is what actually ran.

### How "Couldn't verify" becomes a quality signal

The `unverified` count is the metric that turns epistemic honesty into a measurable product property. A high `claims`-to-`unverified` ratio means the agent found sourceable evidence for most facets; a high `unverified` count flags a target with thin public surface area — useful intelligence in itself.

Tracking `unverified` openly, rather than hiding refusals, keeps the dashboards faithful to the product's actual behavior: an agent that defers to the user when it cannot back a statement. The number cannot be gamed without breaking the "no source, no claim" rule, so it stays an honest signal of run quality.

# 15. Competitive Differentiation & Moat

Why the honesty UX, not the model, is the defensible wedge

Receipts enters a crowded field. Three classes of tool already produce competitive analysis, and a candid reading of each shows where Receipts fits — and, just as importantly, where its advantage is genuinely thin.

### Against generic chatbots (ChatGPT, Claude, Gemini)

A general-purpose chatbot will happily write a competitive teardown on demand. That is precisely the problem product managers describe: confident, well-formatted bullet points with no citations, no dates, and no way to tell which lines are real. The pain is verbatim from r/ProductManagement — "competitive analysis Kabuki-mime theater certainty we don't really have," and "fed ChatGPT release notes… still got hallucinations and went back to reading them all myself."

Receipts differs structurally, not just in tone:

- Cited by construction. The agent must fetch a real page before it asserts anything. The system prompt's hard rule is no source, no claim.
- A visible honesty surface. Anything the agent looked for but could not back lands in an open "Couldn't verify" block instead of being smoothed over.
- Currency. Claims come from the live homepage, pricing page, and changelog at request time — not from training data of unknown vintage.

A chatbot can be prompted to behave better, but it does not ship this contract. Receipts makes the discipline the default and renders it on screen.

### Against enterprise CI suites (Crayon, Klue, Kompyte)

These are real, paid products that continuously index competitors for enterprise competitive-intelligence teams — typically annual contracts in the (illustrative) ~$15k–$30k/year range. They are excellent at what they do, and Receipts does not try to replace them. They are:

- Expensive and enterprise-shaped — procurement, seats, onboarding.
- Continuous, not instant — built to track a roster over time, not to answer "what is this one product, right now, in under a minute."
- Out of reach for the people Receipts serves: solo and early-stage PMs, founders running a pre-build check, and anyone burned by a hallucinated teardown.

Receipts is the accessible, instant wedge for exactly the segment that market structurally ignores. It is free, needs no login, and runs on free, keyless infrastructure (Groq for both models, Jina for search and fetch) — a positioning choice as much as a cost one.

### Against other AI wrappers

The crowded layer is thin LLM wrappers that re-skin a chat box. Most share the chatbot's two gaps: no grounding (no real page fetches) and no honesty surface (nothing that openly admits a gap). Receipts' differentiators are the two things wrappers skip because they are unglamorous — a genuine fetch-then-assert tool loop, and a designed "Couldn't verify" block that treats deferring to the user as a feature, not a failure.

### The honest read on the moat

The defensible wedge is product and positioning, not technology. Every component is replicable — open models, keyless search and fetch, a single API route, an SSE stream. A competent team could rebuild the pipeline in a weekend.

What is harder to copy is the commitment: building the entire interface around epistemic honesty rather than apparent confidence. The detective-dossier design, the Exhibit chips on every claim, the redaction bars over unverifiable items, and the live tool-call loading state all exist to make one promise legible — I am reading real pages right now. Incumbents are positioned around confidence and coverage; none currently owns "honesty as the headline feature."

The moat, then, is a coherent stance plus the discipline to keep the honesty surface visible even when it makes the product look like it knows less. That holds only as long as the team refuses to trade it away for the illusion of completeness — which is precisely the trade competitors keep making.

# 16. Business Model & Go-to-Market

Free, shareable teardowns as the top of funnel; a paid tier scoped honestly as future work

Receipts is a hackathon build, so its business model is deliberately small and honest. Nothing here describes revenue, signups, or traction, because none exists yet. What follows is the path the product is built to support — and the parts of that path it intentionally leaves unbuilt.

### The free product is the funnel

The free experience is the whole product today, and it is also the distribution engine. Anyone can paste a competitor, a hot startup, or their own product, and in under a minute get a six-facet teardown (Exhibits A–F) where every claim links to the source page it came from. Anything the agent cannot back with a fetched source goes into a visible "Couldn't verify" block — shown openly, never fabricated.

The viral mechanic is structural, not bolted on. Every teardown lives at a `/t/<target>` permalink, and that permalink auto-runs the teardown for the next cold visitor — no login, no account, no setup. So sharing a link is not sharing a screenshot of a result; it is handing someone a live tool that demonstrates itself. The ShareBar makes the loop explicit with copy-link, share, and export-to-Markdown actions, and the Markdown export carries every source link with it, so the honesty survives outside the app.

This is also the only persistence in the product. There is no database of saved teardowns and no user-specific history. The permalink is the artifact.

### Distribution

Go-to-market is grounded in where the target users already are and where this specific artifact travels well:

- PM and founder communities. The problem is sourced from real Reddit pain in r/ProductManagement about hallucinated competitive analysis. A tool whose entire pitch is "no source, no claim" is built to be posted into exactly those threads as a direct answer.
- LinkedIn and social shares. A sharp, sourced teardown of a well-known product is inherently shareable; the permalink turns each share into a working demo rather than a static post.
- The permalink loop itself. Each shared `/t/<target>` recruits the next visitor, who can run their own target and share again. This compounding loop — not paid acquisition — is the primary growth assumption.

### A possible future paid tier (not built)

There is an existing paid competitive-intelligence market — Crayon, Klue, Kompyte — generally sold as enterprise contracts (figures around $15k–$30k/year are commonly cited; treat them as illustrative, not a Receipts price point). Those tools continuously index competitors for enterprise CI teams. Receipts deliberately serves the people that market ignores: solo and early-stage PMs, founders running a pre-build check, and anyone burned by a hallucinated teardown.

Taken past the hackathon, a paid tier could plausibly add:

- Saved and tracked competitors — re-running teardowns on a schedule and diffing what changed.
- Team workspaces — shared dossiers instead of one-off permalinks.
- Deeper sourcing — more fetches per run and richer third-party corroboration.

Every one of these is framed as future, not present. Accounts, saved workspaces, a CRM of tracked competitors, and payments were all explicitly cut from the build. That scoping is a feature, not an omission: the brief is to ship one delightful, coherent thing rather than ten half-built ones — which is also what the judging rewards.

### Why the free constraint matters commercially

The 100%-free, no-credit-card stack (Groq for both models, Jina for keyless search and fetch) is an accessibility decision that doubles as a go-to-market one. Zero marginal cost per teardown and zero friction to first value are precisely what let the permalink loop run unthrottled. The same property that makes free distribution viable is the one that would later justify charging for persistence, tracking, and collaboration — rather than for the core teardown itself.

# 17. Roadmap & Future Vision

What we deliberately cut, and what a credible v2 looks like — extending "no source, no claim," not abandoning it.

---

The honest thing to say about a hackathon build is that most of what a competitive-intelligence product could do is missing on purpose. Receipts ships one coherent thing — land, tear down, share — and treats every additional feature as a liability until proven otherwise. This section names what was cut and sketches a v2 that earns its scope, governed by the same rule that governs the agent: nothing is added unless it can show its receipts.

### What is intentionally cut for the hackathon

These were not oversights. Each was removed to protect the one thing the product does well, and to keep a stranger's first run friction-free — no login, no setup, a teardown in under a minute:

- User accounts and auth. No sign-up wall stands between a cold visitor and a teardown. Permalinks (`/t/<target>`) are the only persistence; there is no private account state to manage, secure, or reset.
- Saved workspaces. No personal libraries, folders, or history. A teardown is a shareable artifact, not a document you file away inside a logged-in dashboard.
- A CRM of tracked competitors. Receipts produces a fresh, outside-in snapshot on demand. It does not continuously index a roster of competitors the way Crayon, Klue, or Kompyte do — enterprise CI tools commonly cited in the ~$15k–$30k/year range (illustrative).
- Payments. The stack is free and keyless by design — Groq for both models, Jina for search and fetch. No billing, no plan gate, no credit card. Accessibility is the constraint that shaped the product.

Cutting these is the product thesis in miniature: ship the wedge the incumbents ignore, and refuse to fake the rest.

### What a credible v2 looks like

A second version should extend the honesty principle into time and collaboration — not bolt on enterprise surface area for its own sake. The throughline holds: every new capability still answers "what's the source, and how fresh is it?"

#### Tracked competitors with diffs over time

The natural extension of a single snapshot is a sequence of them. Because every claim already carries a source URL and a confidence level, two teardowns of the same target can be compared structurally — surfacing what a competitor changed: a new pricing tier, a repositioned tagline, a shipped changelog entry. The diff inherits the same discipline: a change is only asserted if both the old and new claims were sourced. "Couldn't verify" stays visible across time, never silently dropped.

### Scheduled re-runs

Tracking implies cadence. A v2 could re-run a teardown on a schedule — weekly, or against a watchlist — and notify the user only when a sourced claim actually changes. Not on noise; only on verifiable movement. This is where Receipts begins to touch the "continuous index" job the enterprise tools own, but from the honest-snapshot direction rather than the opaque-feed one.

### Deeper third-party sourcing

Today the agent aims for one credible third-party review or limitations page — the seam where real weaknesses surface. A v2 could widen that lane: more review aggregators, community threads, and changelog histories, with corroboration (the "high" confidence bar of 2+ sources) carrying more of the load. More sourcing, never more assertion-without-sourcing.

### Team sharing and an API

Permalinks already make a teardown shareable to anyone. Team sharing would add lightweight, attributed collaboration; an API would let a teardown — sources intact — flow into a Notion doc, a PRD, or a CI workflow. Both must carry every source link forward, because a Receipts claim stripped of its receipt is exactly the confabulation the product exists to refuse.

The test for any v2 feature is unchanged: if it can't show its source, it doesn't ship.

## 18. Risks & Mitigations

Where Receipts can break — and the design choices that keep it honest when it does

Receipts makes one strong promise — "No source, no claim." Most of its risks are simply the moments when the real world refuses to cooperate with that promise. Every mitigation below follows the same principle that defines the product: when the agent is uncertain, it lowers its confidence and tells the user, rather than inventing certainty. Honesty degrades gracefully; fabrication does not.

### Page-fetch reliability

The agent's core motion is reading live pages through Jina's reader (`r.jina.ai`). That dependency is also its single biggest source of fragility. JavaScript-heavy sites that render client-side, paywalls, bot-blocking, rate limits, and occasional reader timeouts all mean a given FETCH can come back thin or empty.

Mitigation. The agent never treats a failed fetch as license to guess.

- If a fetch fails or returns too little usable text, the agent falls back to search snippets from `s.jina.ai` for that target — still real, attributable text, just less complete.
- A claim resting on a single snippet is marked medium confidence, so the amber highlighter (reserved for "high" confidence) only lands on facts corroborated by two or more sources.
- The claim still carries its source link, so the user can click through and see exactly what the agent saw.
- If even snippets can't support a point, it is routed to the visible "Couldn't verify" block. The hard rule in the system prompt holds: no fetched source means the assertion is not made.

Fetch failures shrink the teardown or soften its confidence. They never silently corrupt it.

### Ambiguous product names

"Notion" or "Linear" is unambiguous. A common word, an internal codename, or a name shared by several products is not — and the agent could tear down the wrong company.

Mitigation. The agent states the match it chose: the canonical URL and tagline it locked onto appear in the output header, so the user can see at a glance whether it found the right target. If it picked wrong, one re-run lets them disambiguate. This is consistent with the human-in-the-loop posture throughout Receipts — the agent surfaces its interpretation instead of hiding it.

### Thin public information

Some targets — pre-launch startups, stealth tools, products with sparse public pages — simply don't have enough fetchable material to fill all six exhibits. A dishonest tool would pad the gaps.

Mitigation. Receipts uses an honest empty state. Facets with no sourced claims render empty, and items that were searched but unbacked land in "Couldn't verify" with a reason where one exists. A short, sourced teardown that admits what it doesn't know is worth more than a long, confident one that's half-invented — and it demonstrates the product's thesis directly.

### Free-tier and latency limits

Receipts runs entirely on free infrastructure by design — accessibility is the point — which means real ceilings: Groq free-tier limits on both models, Jina keyless caps on search and fetch, and, for the hackathon's analytics, Novus's 500-prompts-per-month allowance. Batched or hourly analytics processing can also delay when dashboard data appears.

Mitigation.

- The model-agnostic text protocol allows a swap to Google Gemini's free tier via three env vars (`LLM_BASE_URL` / `LLM_MODEL` / `LLM_API_KEY`) if Groq limits bite during judging.
- With no key at all, the app serves a bundled real Linear sample, so the full UI and the "Couldn't verify" doctrine stay demonstrable even under a quota outage.
- For Novus's prompt cap and batch latency, the plan is to seed real traffic early — run teardowns well ahead of the deadline so the Conversations, Tools-used, and Models-used dashboards are populated and screenshot-ready, rather than racing the cap on submission day.

Across all four risks, the failure mode is the same, and intentional: Receipts gets quieter and more cautious, never louder and more wrong.

## 19. Hackathon Fit: Mapping to the Judging Rubric

How Receipts earns marks on each 25% axis — and which judge each one speaks to

World Product Day 2026 scores every entry on four equally weighted axes — Product Thinking, Craft & Execution, Originality & Ambition, and Shippedness — each worth 25%. The organizers state plainly that beautiful execution of a simple idea beats an ambitious-but-broken one. Receipts was scoped against exactly that sentence. Below, each axis is tied to the documented philosophy of the judge most likely to weigh it.

### Product Thinking (25%)

The problem is sourced, not assumed. The pain is verbatim from r/ProductManagement (2024–2026): AI competitive analysis as "confabulated fairy tales" and "Kabuki-mime theater certainty," with PMs giving up and reading release notes by hand. The persona is named and narrow — solo and early-stage PMs, founders running a pre-build check, anyone burned by a hallucinated teardown. The adjacent market is real and paid: Crayon, Klue, and Kompyte sell continuous competitive intelligence on enterprise contracts (commonly cited around $15k–$30k per year — treat as illustrative). Receipts is the accessible wedge those tools ignore. This axis speaks directly to Amit Godbole, who looks for willingness-to-pay and problems people already pay to solve.

### Craft & Execution (25%)

Receipts is one coherent artifact, not a feature pile. The Dossier design system — a clay-and-paper case file in Fraunces and Inter, receipt-stamp "Exhibit" chips on every claim, an amber highlighter that lands only on corroborated claims, redaction bars over the "Couldn't verify" block, and a loading state that surfaces the agent's real tool calls live — all serves the single idea. Copy is deliberate: "No source, no claim," Exhibits A–F, "Couldn't verify." Deliberately cut: accounts, saved workspaces, a tracked-competitor CRM, payments. This is the axis for Dave Killeen (taste and ruthless scope over feature count) and Valeria Khokhlova (product storytelling) — and it honors the rubric's own value by being neither rough nor overscoped.

### Originality & Ambition (25%)

The wedge is epistemic honesty as the product itself. The agent trades hallucinated confidence for verifiable, current intel and visibly refuses to assert anything it cannot source — the hard system-prompt rule "no source  not asserted," surfaced as a separate "Couldn't verify" block. That block doubles as the human-in-the-loop gesture: the agent defers judgment to the user rather than auto-asserting. No incumbent owns this position. It maps to Joe Dreimann, who wrote about trust in AI and the principle that nothing reaches production without approval, and to Curtis Michelson, who rewards rigor over innovation theater. Receipts makes "showing your work" the feature.

### Shippedness (25%)

This axis separates a demo from a product, and Receipts is built to clear it:

- No-login public URL. A stranger pastes a target and gets a live teardown; `/t/<target>` permalinks auto-run for cold visitors.
- Real, tool-rich agent runs. Each session makes 3–5 visible FETCHes across an official site, a pricing page, a changelog, and a third-party review — populating the Novus Conversations, Tools-used, and Models-used dashboards with genuinely agentic traffic.
- Goal completions. The funnel runs landed → submitted_target → teardown_rendered → teardown_shared_or_exported, with the share/export event as the tracked goal.

Novus instrumentation is mandatory — projects without it are ineligible — and Receipts treats it as first-class, not bolted on. This is the axis for Sean Ryan, who rewards genuine end-to-end shippedness. Across all four axes, one disciplined build does real work for a real, sourced problem.

# 20. Conclusion & The Bigger Bet

Receipts is a small product making a large argument about what useful AI should be.

## The small product

Receipts does one thing. You paste a product — a competitor, a hot startup, your own — and a single AI agent reads its real homepage, pricing page, and changelog, plus at least one credible third-party review. In under a minute it returns a six-facet teardown across Exhibits A through F (Positioning, ICP, Pricing, Recent moves, Strengths, Weaknesses), where every claim links to the exact source page it came from. Anything the agent looked for but could not back with a fetched source goes into a visible "Couldn't verify" block — shown openly, never invented.

That is the whole product. No accounts, no saved workspaces, no CRM of tracked competitors, no payments. Permalinks are the only persistence. The scope was cut this hard on purpose, so the one thing could be done well.

## The large argument

The argument underneath that small product is bigger than competitive analysis.

Most AI tools compete on confidence. They produce fluent, well-formatted certainty — and product managers have learned, the hard way, that the certainty is often hollow. The pain is documented in their own words: "confabulated fairy tales," "competitive analysis Kabuki," feeding a model the release notes and still getting hallucinations. The response has been to stop trusting the tools and go back to reading the pages by hand.

Receipts bets the other direction. The future of useful AI is not more confident output — it is more honest, grounded, citable output. An agent earns trust not by sounding sure, but by showing its work and visibly declining to assert what it cannot source.

### "No source, no claim" as a stance

"No source, no claim" is the rallying line, but it is meant as more than a tagline. It is a position the field will eventually have to take:

- A claim without a source is a guess wearing a suit. Fluency is not evidence. Receipts routes an unsourced statement to the "Couldn't verify" block rather than asserting it.
- Refusal is a feature, not a failure. When the agent declines to claim something, it hands judgment back to the person — the human-in-the-loop gesture made concrete.
- Honesty is checkable. High-confidence claims are corroborated by two or more sources; every link survives the Markdown export, so a skeptical reader can audit the teardown instead of trusting it.

This position is currently unowned. The established competitive-intelligence tools — Crayon, Klue, Kompyte — sell enterprise contracts (often cited in the roughly $15k–$30k/year range, illustrative only) and index competitors continuously for large CI teams. The newer AI tools optimize for confident bullet points. Neither has planted a flag on epistemic honesty. Receipts is the accessible, instant, honest wedge for the people that market ignores: solo and early-stage PMs, founders running a pre-build check, and anyone who has been burned by a hallucinated teardown.

## What a stranger can do right now

The bet is not a slide. It is a deployed URL with no login, where a stranger can act immediately:

- Run a teardown. Paste any product and watch the agent's real tool calls stream live — SEARCHING → FETCHING → SYNTHESIZING — so the core claim ("I am reading real pages right now") is visible, not promised.
- Check the receipts. Click any Exhibit chip to open the source page behind a claim, and read the "Couldn't verify" block to see exactly what the agent refused to assert.
- Share or export. Open a `/t/<target>` permalink that auto-runs the teardown for a cold visitor, or export to Markdown with every source link intact.

That is the bigger bet in one sentence: build the smallest honest version of a tool people already pay for, ship it where anyone can use it, and let the citations make the argument. No source, no claim.